

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Organización de Lenguajes y Compiladores 2



Manual Técnico - Codex Latinus

Registro académico: 201831260
Nombre: Michael Kristopher Marín Reyes

Quetzaltenango, Agosto de 2026.

Índice

Descripción.....	3
1. Stack Tecnológico.....	3
2. Archivo de Configuración.....	3
3. Requisitos del Sistema y Entorno de Desarrollo.....	6
4. Compilación y Generación del Ejecutable.....	6
Opción A: Usando Maven.....	6
Opción B: Usando IntelliJ IDEA.....	7
5. Arquitectura General del Sistema.....	7
A. Definición de la Gramática (antlr4).....	7
B. Capa de Presentación (frontend/ - Java Swing & FlatLaf).....	7
C. Capa de Procesamiento y Compilación (backend/).....	8
D. Punto de Entrada (Codex_latinus.java).....	8
6. Gramática.....	8
6.1. Estructura Raíz y Programa Principal.....	8
6.2. Definición de Tipos y Estructuras (Structs).....	9
6.3. Declaración de Variables y Arreglos Globales/Locales.....	9
6.4. Módulo de Funciones y Procedimientos (Munera).....	10
6.5. Expresiones y Operaciones.....	10
6.6. Bloque Principal y Sentencias de Ejecución.....	11
6.7. Control de Flujo y Ciclos.....	11
6.8. Lógica Booleana y Relacional.....	11
7. Diagrama de clases.....	12

Manual Técnico: Codex Latinus

Descripción

Este documento detalla las especificaciones técnicas, el stack tecnológico utilizado, los requisitos de configuración y el proceso de compilación e instalación de la aplicación de escritorio **Codex Latinus**.

1. Stack Tecnológico

El proyecto está desarrollado enteramente bajo un entorno de arquitectura modular en Java, integrando herramientas estándar de la industria para el análisis léxico, sintáctico y el diseño de interfaces gráficas:

- **Lenguaje: Java 22 (JDK 22)** → Utilizado para aprovechar las características modernas del lenguaje, gestión de memoria y el rendimiento del motor de ejecución.
- **Analizador Léxico y Sintáctico: ANTLR 4 (ANother Tool for Language Recognition)** — Empleado para la definición de la gramática del lenguaje (.g4), generación automática del *Lexer*, *Parser* y el recorrido del Árbol de Análisis Sintáctico (AST) mediante el patrón *Visitor*.
- **Interfaz Gráfica de Usuario (GUI): Java Swing** → Framework nativo de Java para la construcción de la interfaz de escritorio, paneles de edición de código, consola de resultados y visualización de tablas de símbolos y errores.
- **Librería de Apariencia (Look and Feel): FlatLaf** → Integrada para dotar a la interfaz de Java Swing de un diseño moderno, limpio y profesional (compatible con temas oscuros y claros), elevando la experiencia visual del IDE.

2. Archivo de Configuración

A continuación se presenta la estructura completa del archivo de configuración utilizada en el proyecto para asegurar la correcta compilación y resolución de dependencias:

```
<?xml version="1.0" encoding="UTF-8"?>

<project                                xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

                                xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>compi2</groupId>
```

```
<artifactId>codex_latinus</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>
<dependencies>
  <dependency>
    <groupId>org.netbeans.external</groupId>
    <artifactId>AbsoluteLayout</artifactId>
    <version>RELEASE260</version>
  </dependency>
  <dependency>
    <groupId>com.formdev</groupId>
    <artifactId>flatlaf</artifactId>
    <version>3.6</version>
  </dependency>
  <dependency>
    <groupId>com.formdev</groupId>
    <artifactId>flatlaf-intellij-themes</artifactId>
    <version>3.6</version>
  </dependency>
  <dependency>
    <groupId>org.antlr</groupId>
    <artifactId>antlr4</artifactId>
    <version>4.13.2</version>
  </dependency>
</dependencies>
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
```

```
<maven.compiler.release>22</maven.compiler.release>

<exec.mainClass>codex_latinus.Codex_latinus</exec.mainClass>
</properties>
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.13.0</version>
      <configuration>
        <release>22</release>
      </configuration>
    </plugin>
    <plugin>
      <groupId>org.antlr</groupId>
      <artifactId>antlr4-maven-plugin</artifactId>
      <version>4.13.2</version>
      <executions>
        <execution>
          <id>antlr</id>
          <phase>generate-sources</phase>
          <goals>
            <goal>antlr4</goal>
          </goals>
        </execution>
      </executions>
      <configuration>
        <visitor>true</visitor>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>
```

```
<listener>true</listener>

</configuration>

</plugin>

</plugins>

</build>

</project>
```

3. Requisitos del Sistema y Entorno de Desarrollo

Para compilar, modificar o ejecutar el código fuente de la aplicación, el equipo de desarrollo o despliegue debe contar con:

- **Java Development Kit (JDK 22):** Asegúrate de tener instalado JDK 22 y configurada la variable de entorno `JAVA_HOME` apuntando a su ruta de instalación. Puedes validarlo en la terminal con:

```
java -version
```

- **Entorno de Desarrollo Recomendado:** IntelliJ IDEA (con soporte nativo para proyectos Java y plugins de ANTLR) o Apache Netbeans.
- **Gestor de Dependencias:** Maven o Gradle (según la configuración del proyecto para manejar las librerías de ANTLR runtime).

4. Compilación y Generación del Ejecutable

Para empaquetar la aplicación de escritorio junto con todas sus dependencias (incluyendo ANTLR) en un único archivo ejecutable:

Opción A: Usando Maven

1. Abre la terminal en la raíz del proyecto.
2. Ejecuta el comando de empaquetado (asegurándote de tener configurado el plugin *Maven Shade*):

```
mvn clean package
```

3. El archivo ejecutable se generará en la ruta: `target/codex_latinus.jar`.

Opción B: Usando IntelliJ IDEA

1. Ve a **File > Project Structure > Artifacts**.
2. Añade un nuevo artefacto de tipo **JAR > From modules with dependencies...**
3. Selecciona la clase principal (Main) que inicializa la interfaz de Java Swing.
4. Selecciona **Build > Build Artifacts...** y compila. El archivo estará disponible en la carpeta `out/artifacts/`.

Para ejecutar la aplicación empaquetada desde cualquier terminal compatible con Java 22, utiliza:

```
java -jar codex_latinus.jar
```

5. Arquitectura General del Sistema

El software está estructurado bajo una arquitectura limpia y modular dividida en tres capas principales: la definición formal de la gramática, la interfaz gráfica de usuario (**Frontend**) y el motor de procesamiento, análisis e interpretación (**Backend**).

A. Definición de la Gramática (antlr4/)

- **Codex_latinus.g4**: Contiene las reglas léxicas y sintácticas formales del lenguaje escritas para ANTLR 4, sirviendo de base para la generación automática del lexer, el parser y los árboles de derivación.

B. Capa de Presentación (frontend/ - Java Swing & FlatLaf)

Gestiona la interfaz gráfica de escritorio y la interacción con el usuario, integrando componentes visuales avanzados:

- **MainWindow**: Ventana principal contenedora de los paneles de navegación, menús y pestañas de trabajo.
- **EditorPanel**: Editor de código fuente con soporte para numeración de líneas (LineNumberComponent).
- **Paneles de Análisis y Depuración**:
 - **ParserTreePanel**: Visualizador gráfico del Árbol de Análisis Sintáctico (AST).
 - **SymbolTablePanel**: Tabla interactiva para la supervisión de símbolos y ámbitos.
 - **ErrorTablePanel**: Registro detallado de errores léxicos, sintácticos y semánticos.
 - **StackVisualizerPanel**: Interfaz de seguimiento de la pila de procesos (StackState y StackVisualizerListener).

C. Capa de Procesamiento y Compilación (backend/)

Núcleo lógico encargado de transformar, validar y ejecutar el código fuente a través de subsistemas especializados:

- **Compilación y Utilidades:**
 - Compiler.java y DotGenerator.java: Orquestadores del proceso de compilación y generación de gráficos estructurales.
 - TypeChecker (en utils/): Validador estricto de tipos de datos y reglas de inferencia implícita.
- **Manejadores Semánticos (handlers/):**
 - Subsistemas especializados para la modularidad del análisis: AssignmentHandler, ConditionalHandler, ControlFlowHandler, DeclarationHandler, FunctionHandler, LoopHandler y StructHandler.
- **Sistema de Símbolos y Ámbitos (symbols/ y stack/):**
 - Estructuras de control de alcance (Scope, SymbolTable, Symbol) y tipado (TypeTable, TypeInfo), junto al control de estado de ejecución en pila (StackState).
- **Visitantes del AST (visitors/):**
 - SemanticAnalyzerVisitor: Recorre el árbol para verificar tipos, validar declaraciones y poblar la tabla de símbolos.
 - InterpreterVisitor: Ejecuta secuencialmente las instrucciones del lenguaje sobre el ámbito actual (asignaciones, estructuras de control, llamadas y retornos con redere).
 - PigLatinVisitor: Transformador alternativo de código según las especificaciones del lenguaje.
- **Control de Errores (errors/):**
 - CompilationError y CustomErrorListener para la captura y gestión robusta de excepciones de compilación.

D. Punto de Entrada (Codex_latinus.java)

- Clase principal que inicializa el sistema, configura los temas gráficos e inicia la ejecución de la aplicación de escritorio.

6. Gramática

La gramática formal de **Codex Latinus** está estructurada mediante reglas sintácticas (No Terminales) definidas en formato ANTLR4. A continuación, se detalla el funcionamiento y propósito específico de cada producción que compone el analizador sintáctico:

6.1. Estructura Raíz y Programa Principal

- **init: codex_latinus;**

- *Propósito:* Es la regla de inicio absoluta del compilador. Actúa como el punto de entrada que redirige el flujo de análisis hacia la regla estructural principal del lenguaje.
- **codex_latinus: (structura_def | variables | munera)* maior;**
 - *Propósito:* Define la macro-estructura de un archivo fuente. Permite la declaración opcional y repetida (*) de estructuras de datos (structura_def), bloques de variables globales (variables) o bloques de funciones (munera), exigiendo de forma obligatoria que el archivo culmine con el bloque de ejecución principal (maior).

6.2. Definición de Tipos y Estructuras (Structs)

- **structura_def: STRUCTURA VARIABLE LLAVE_IZQ miembro_structura* LLAVE_DER FINIS PUNTO_COMA;**
 - *Propósito:* Permite definir tipos de datos personalizados compuestos. Requiere la palabra clave structura, un identificador (VARIABLE), un cuerpo delimitado por llaves que contiene cero o más miembros, y finaliza con la palabra clave de cierre (finis) y un punto y coma.
- **miembro_structura:** Define los campos internos que conforman una estructura:
 - *Opción 1:* Un campo escalar individual prefijado por esto, un nombre, dos puntos y un tipo de dato primitivo o el nombre de otra estructura.
 - *Opción 2:* Un campo de arreglo dinámico prefijado por series.
 - *Opción 3:* Un campo de arreglo de tamaño fijo acotado mediante corchetes ([NUMERO_ENTERO]).

6.3. Declaración de Variables y Arreglos Globales/Locales

- **variables: VARIABLES MAYOR_QUE (declaracion | arreglo_declaracion | estructura_def)+;**
 - *Propósito:* Bloque global de declaración de variables. Inicia con la palabra clave VARIABLES seguida de un operador mayor que (>) y exige al menos una (+) declaración de variables, arreglos o estructuras anidadas.
- **declaracion:** Define variables individuales permitiendo inferencia o asignación explícita de tipos primitivos (numerus, decimalis, verum, falsus), cadenas de texto (textum), caracteres (littera), instanciaciones de estructuras o expresiones aritméticas/lógicas generales.
- **arreglo_declaracion:** Permite inicializar series de datos de tamaño fijo especificando el número de elementos entre corchetes, su tipo y un bloque opcional de valores literales entre llaves ({...}).
- **elemento_arreglo / elemento_arreglo_struct:** Listas separadas por comas que contienen expresiones evaluables o instanciaciones de estructuras para poblar arreglos de manera literal.
- **structura_instanciacion:** Facilita la creación de instancias de estructuras asignando valores a sus atributos mediante un bloque entre llaves, o bien accediendo a posiciones específicas de un arreglo de estructuras mediante corchetes.
- **atributo_asignacion:** Define la asignación de un valor específico a una propiedad interna de una estructura durante su instanciación (VARIABLE [:] valor).

- **arreglo_literal**: Representa una colección de valores literales encerrados entre llaves.

6.4. Módulo de Funciones y Procedimientos (Munera)

- **munera**: **MUNERA MAYOR_QUE funcion+**;
 - *Propósito*: Agrupa la sección de declaraciones subrutinarias del lenguaje. Inicia con **MUNERA >** y requiere obligatoriamente uno o más bloques de funciones.
- **funcion**: Regla general que ramifica la subrutina en dos variantes: funciones con retorno (**ratio_funcion**) o procedimientos sin retorno (**actio_funcion**).
- **ratio_funcion**: **RATIO tipo_dato VARIABLE PARENTESIS_IZQ parametros? PARENTESIS_DER LLAVE_IZQ variables_locales? sentencia* reddere_sentencia LLAVE_DER FINIS PUNTO_COMA;**
 - *Propósito*: Define una función con retorno estricto. Requiere la palabra clave **RATIO**, el tipo de dato que devolverá, un identificador, parámetros opcionales, variables locales opcionales, un conjunto de sentencias y **exige** de forma obligatoria la sentencia de retorno **reddere_sentencia**.
- **actio_funcion**: **ACTIO VARIABLE PARENTESIS_IZQ parametros? PARENTESIS_DER LLAVE_IZQ variables_locales? sentencia* LLAVE_DER FINIS PUNTO_COMA;**
 - *Propósito*: Define un procedimiento procedural (equivalente a una función void). Utiliza la palabra clave **ACTIO**, un identificador, parámetros, variables locales opcionales y sentencias, sin requerir valor de retorno.
- **tipo_dato**: Especifica los tipos primitivos soportados: **numerus** (enteros), **textum** (textos), **decimalis** (decimales), **littera** (caracteres), **verum y falsus** (booleanos).
- **parametros / parametro**: Administran la recepción de argumentos de entrada en las funciones bajo la sintaxis esto **[nombre] : [tipo]**.
- **variables_locales / declaracion_local**: Permiten declarar un ámbito de variables exclusivas dentro de una función, delimitadas por la palabra clave **VARIABLES** y corchetes [...].

6.5. Expresiones y Operaciones

- **expresion**: **termino (operacion_aritmetica termino)*;**
 - *Propósito*: Resuelve operaciones aritméticas encadenadas evaluando un término base seguido de cero o más combinaciones de operadores aritméticos y términos sucesivos.
- **termino**: Representa las unidades atómicas que pueden formar parte de una expresión: variables, accesos a propiedades de estructuras, literales numéricos, cadenas, caracteres, booleanos o llamadas a funciones.
- **acceso_miembro / arreglo_acceso**: Gestionan la navegación profunda en estructuras de datos anidadas y arreglos mediante operadores de punto (.) o índices de corchetes ([...]).
- **operador_aritmetica**: Define los operadores matemáticos binarios básicos: suma (+), resta (-), multiplicación (*) y división (/).

6.6. Bloque Principal y Sentencias de Ejecución

- **maior: MAIOR MAYOR_QUE sentencia* FINIS PUNTO_COMA;**
 - *Propósito:* Es el bloque de ejecución principal (equivalente a la función main en otros lenguajes). Inicia con MAIOR >, ejecuta secuencialmente cero o más sentencias y concluye estrictamente con finis;.
- **sentencia:** Regla polimórfica que centraliza todas las acciones ejecutables permitidas dentro del flujo imperativo: impresiones, lecturas, asignaciones, condicionales, ciclos, saltos, llamadas a funciones y operaciones de incremento/decremento unario (++ , --).
- **leer_sentencia (<<):** Captura datos desde la consola hacia una variable o miembro específico.
- **asignacion_sentencia (=):** Almacena el resultado de una expresión, condición, instanciación o arreglo literal dentro de una variable o propiedad de estructura.
- **imprimir_sentencia (>>):** Envía flujos de texto, valores de variables, resultados de funciones o accesos a miembros hacia la salida estándar de consola.

6.7. Control de Flujo y Ciclos

- **si_sentencia:** Implementa la estructura condicional clásica si (condicion) { ... } aliter { ... } finis;, soportando además bloques alternos múltiples (aliter_bloque*).
- **aliter_bloque:** Gestiona las bifurcaciones condicionales secundarias de tipo *else-if* (aliter (condicion) { ... }).
- **ciclo_dum:** Bucle condicional pre-evaluado de tipo *while* (dum (condicion) { ... } finis;).
- **ciclo_facere:** Bucle post-evaluado de tipo *do-while* (facere { ... } dum (condicion);).
- **ciclo_per:** Bucle iterativo indexado de tipo *for* (per (inicializacion; condicion; incremento) { ... }). Sus subreglas (inicializacion_per, condiciones_per, incremento_per) controlan la declaración de la variable de índice, la evaluación de continuidad y el paso iterativo.
- **salto_sentencia:** Controla las interrupciones de bucles mediante perge; (equivalente a *continue*) o interrumpe; (equivalente a *break*).

6.8. Lógica Booleana y Relacional

- **condicion / conjuncion / negacion_logica / primaria_logica:** Forman la jerarquía de precedencia para evaluar expresiones booleanas complejas combinando operadores lógicos de disyunción (OR / ||), conjunción (AND / &&), negación (non / !) y comparaciones relacionales.
- **operador_relacional:** Define los operadores de comparación entre expresiones: mayor o igual (>=), menor o igual (<=), igualdad estricta (==), desigualdad (!=), menor que (<) y mayor que (>).
- **llamada_funcion / argumentos:** Permiten invocar subrutinas pasando un identificador de función y una lista de expresiones separadas por comas enviadas como argumentos de entrada.

7. Diagrama de clases

Adjunto a este documento podrá encontrar la imagen en formato svg y png, donde se puede visualizar de mejor manera cada clase y sus relaciones.